



第5章 递归

提纲
CONTENTS

5.1 什么是递归

5.2 栈和递归

5.3 递归算法的设计



5.1 什么是递归

5.1.1 递归的定义

在定义一个过程或函数时，出现直接或者间接调用自己的成分，称之为**递归**。

- 若直接调用自己，称之为**直接递归**。
- 若间接调用自己，称之为**间接递归**。



【例5.1】根据求阶乘的定义，给出求 $n!$ （ n 为正整数）的递归函数。

解

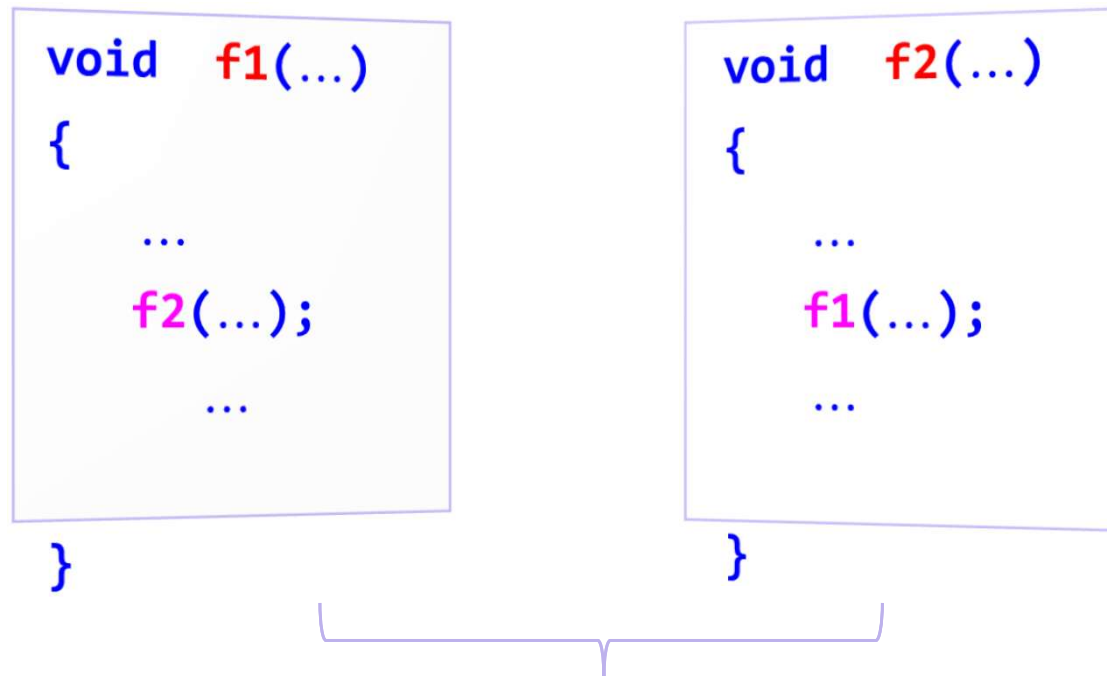
- 求 $n!$ 的定义是 $1!=1$ ， $n!=n*(n-1)!$ 。
- 递归函数 $\text{fact}(n)$ 如下。

```
int fact(int n)
{  if (n==1)                //语句1
    return 1;               //语句2
    else                    //语句3
    return n*fact(n-1);      //语句4
}
```

直接递归函数



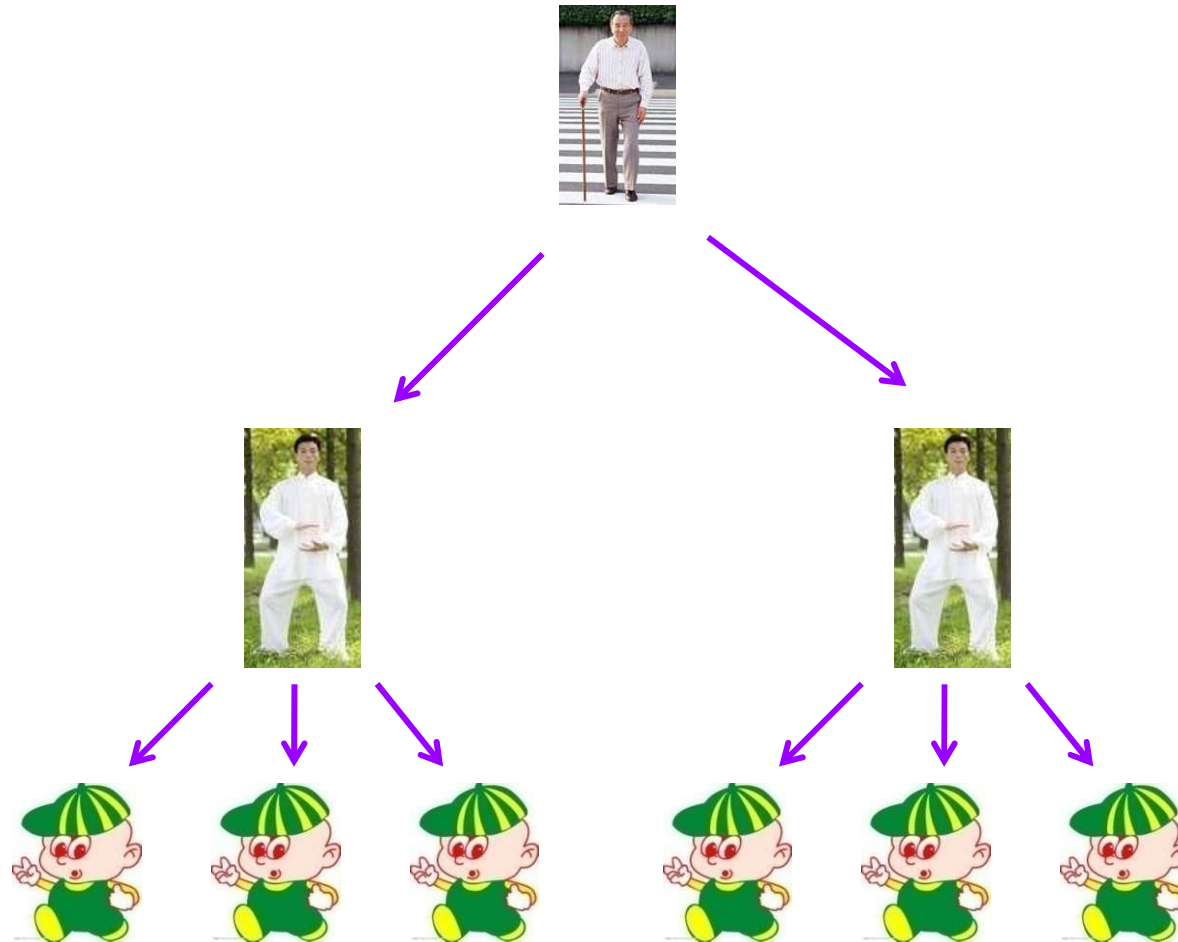
间接递归示例：



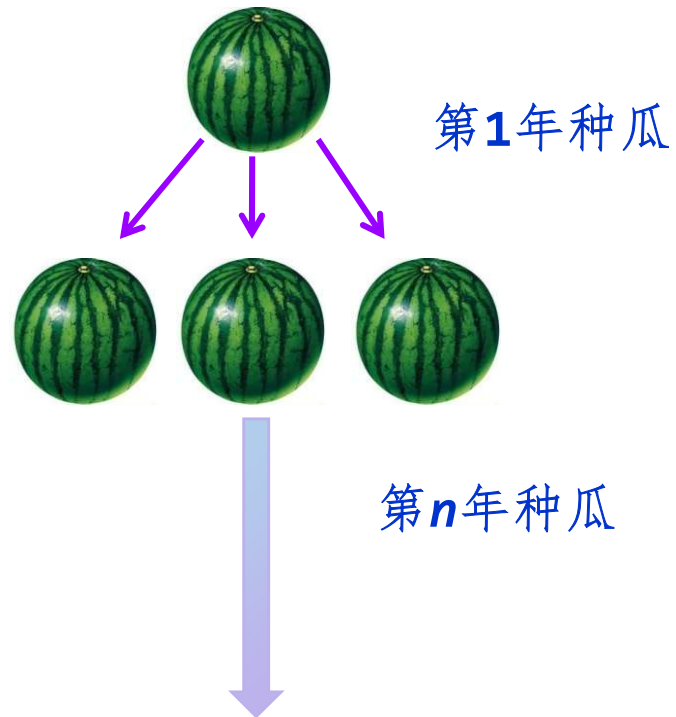
总可以转换为直接递归函数

递归：无处不在

实例1：家谱



实例2：种瓜得瓜





实例3：二进制数





5.1.2 何时使用递归

在以下三种情况下，常常要用到递归的方法。

1、定义是递归的

有许多数学公式、数列等的定义是递归的。

例如，求 $n!$ 和Fibonacci数列等。这些问题的求解过程可以将其递归定义直接转化为对应的递归算法。



思考题：

请你给出正整数的定义。



- **1**是正整数。
- 如果 **n** 是正整数，则 **$n+1$** 也是正整数。



2、数据结构是递归的

有些数据结构是递归的。例如，第2章中介绍过的单链表就是一种递归数据结构，其结点类型定义如下：

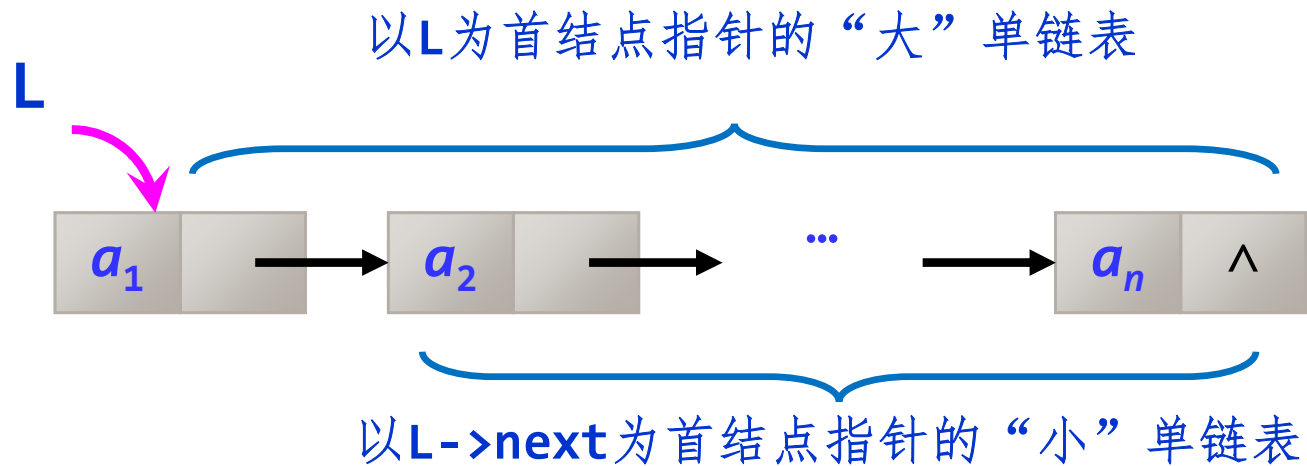
```
typedef struct LNode
{
    ElemType data;
    struct LNode *next;
} LinkNode;
```

————→ 指向同类型结点的指针

↓
递归数据结构



不带头结点单链表示意图



体现出这种单链表的递归性。

思考：如果带有头结点又会怎样呢？？？

3、问题的求解方法是递归的

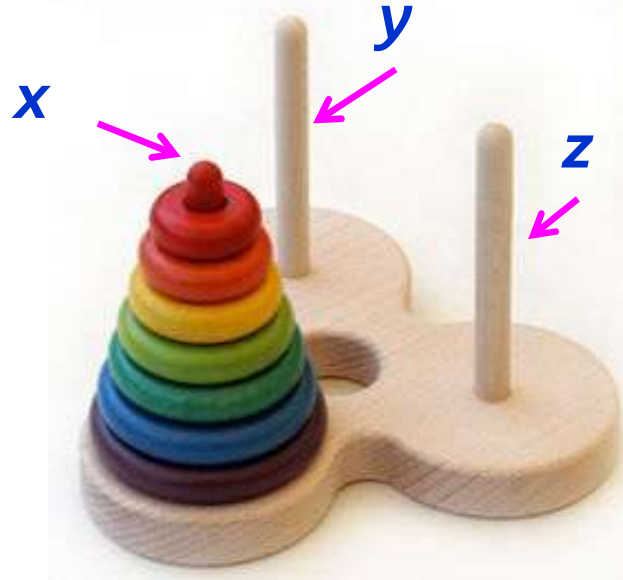
Hanoi问题： X 、 Y 和 Z 的塔座，在塔座 X 上有 n 个直径各不相同，从小到大依次编号为 $1 \sim n$ 的盘片。要求将 X 塔座上的 n 个盘片移到塔座 Z 上。



移动规则：

- 每次只能移动一个盘片；
- 盘片可以插在 X 、 Y 和 Z 中任一塔座上；
- 任何时候都不能将一个较大的盘片放在较小的盘片上方。

- 设 $\text{Hanoi}(n, x, y, z)$ 表示将 n 个盘片从 x 通过 y 移动到 z 上。



$\text{Hanoi}(n, x, y, z)$



$\text{Hanoi}(n-1, x, z, y);$

$\text{move}(n, x, z)$: 将第 n 个圆盘从 x 移到 z ;

$\text{Hanoi}(n-1, y, x, z)$

“大问题”转化为若干个“小问题”求解



5.1.3 递归模型

递归模型是递归算法的抽象，它反映一个递归问题的递归结构。
例如求 $n!$ 递归算法对应的递归模型如下：

$\text{fun}(1)=1$ (1)

$\text{fun}(n)=n*\text{fun}(n-1)$ $n>1$ (2)

递归出口

递归体

一般地，一个递归模型是由递归出口和递归体两部分组成。

- 递归出口确定递归到何时结束。
- 递归体确定递归求解时的递推关系。



递归出口的一般格式如下：

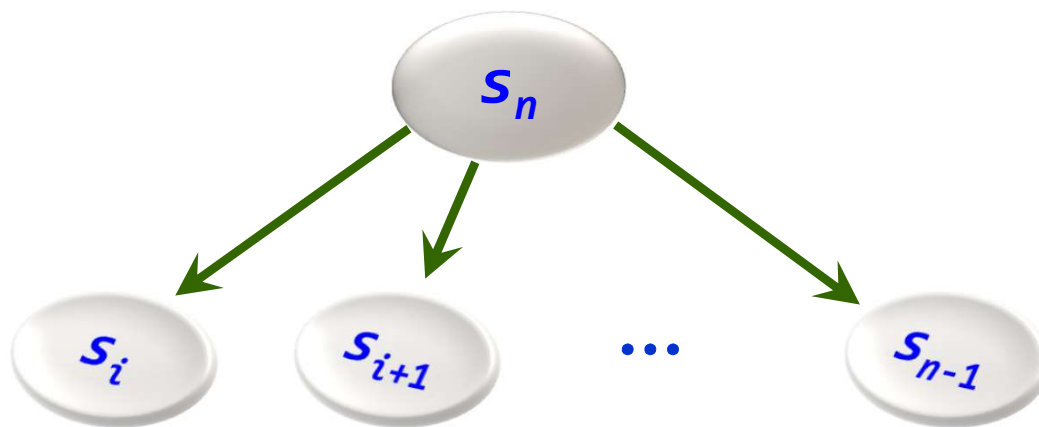
$$f(s_1) = m_1$$

这里的 s_1 与 m_1 均为常量，有些递归问题可能有几个递归出口。

递归体的一般格式如下：

$$f(s_n) = g(f(s_i), f(s_{i+1}), \dots, f(s_{n-1}), \underbrace{c_j, c_{j+1}, \dots, c_m}_{\text{常量}})$$

g 是一个非递归函数



大问题求解

转化

若干个相似子问题求解



递归思路

把一个不能或不好直接求解的“**大问题**”转化成几个或几个“**小问题**”来解决；

再把这些“**小问题**”进一步分解成更小的“**小问题**”来解决。



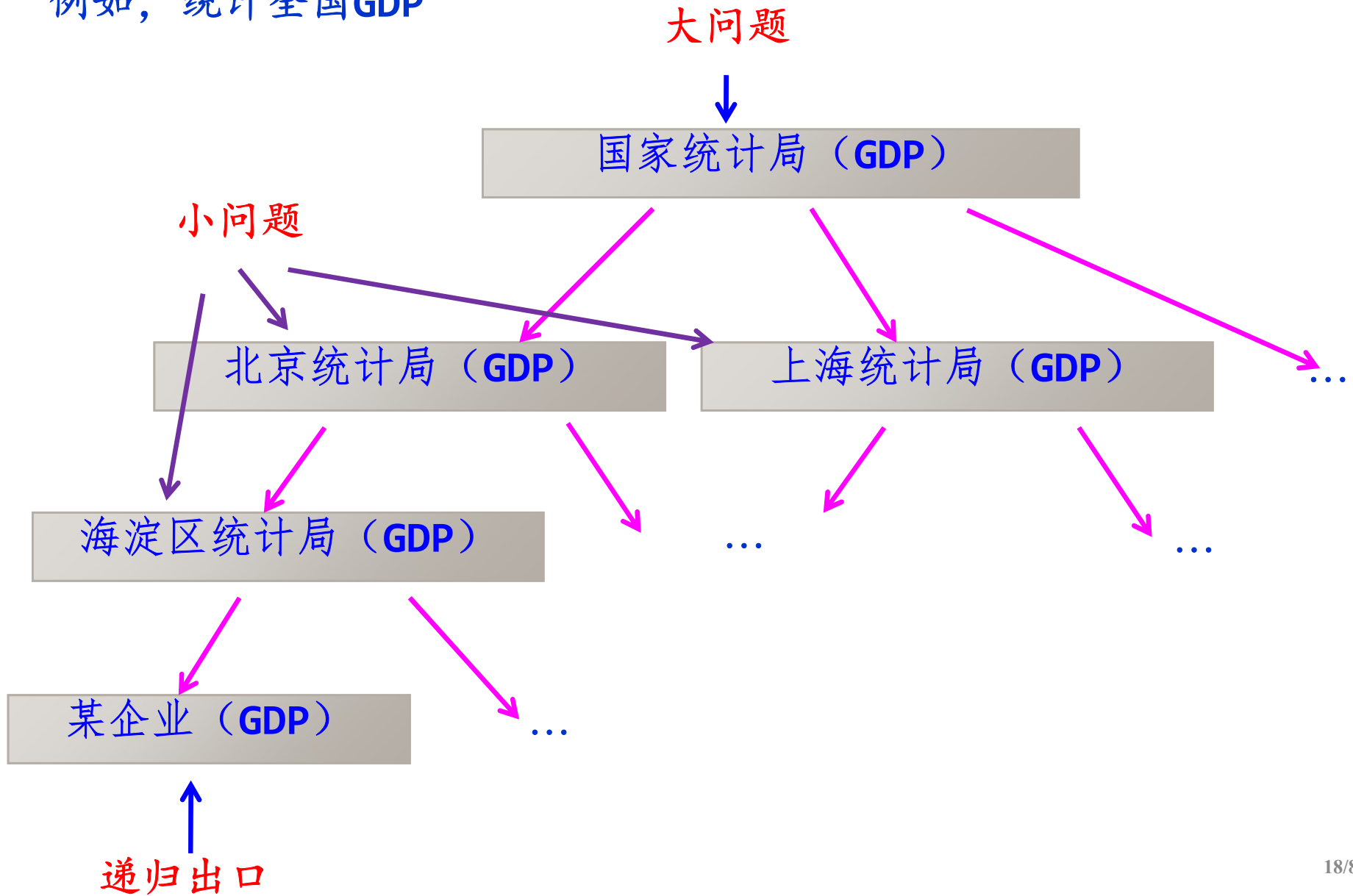
直到

每个“小问题”都可以直接解决（此时分解到递归出口）

但递归分解不是随意的分解，递归分解要**保证“大问题”与“小问题”相似**，即求解过程与环境都相似。



例如，统计全国GDP





为了讨论方便，简化上述递归模型为：

$$f(s_1)=m_1$$

$$f(s_n)=g(f(s_{n-1}), c_{n-1})$$

求 $f(s_n)$ 的分解过程如下：

$$f(s_n)$$



$$f(s_{n-1})$$



...



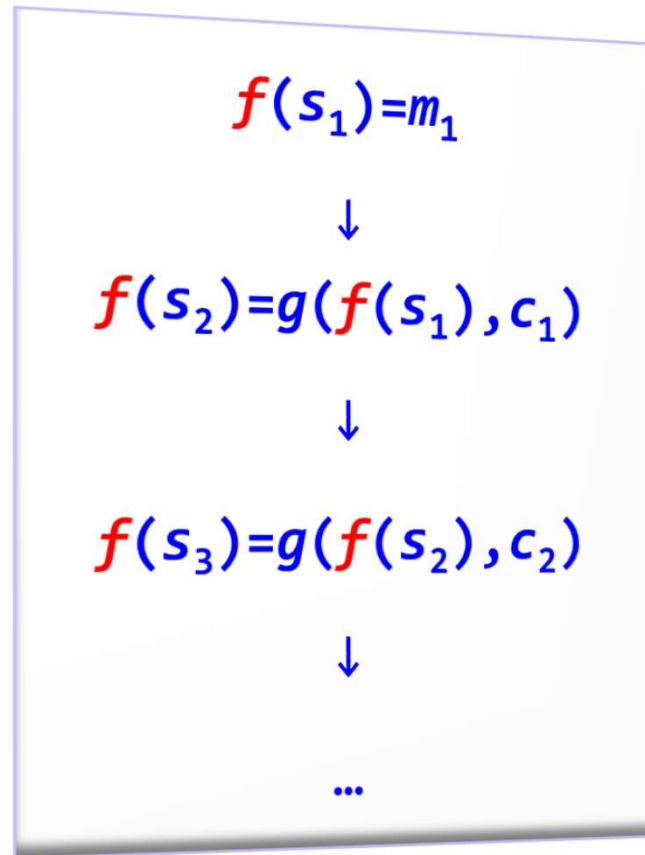
$$f(s_2)$$



$$f(s_1)$$



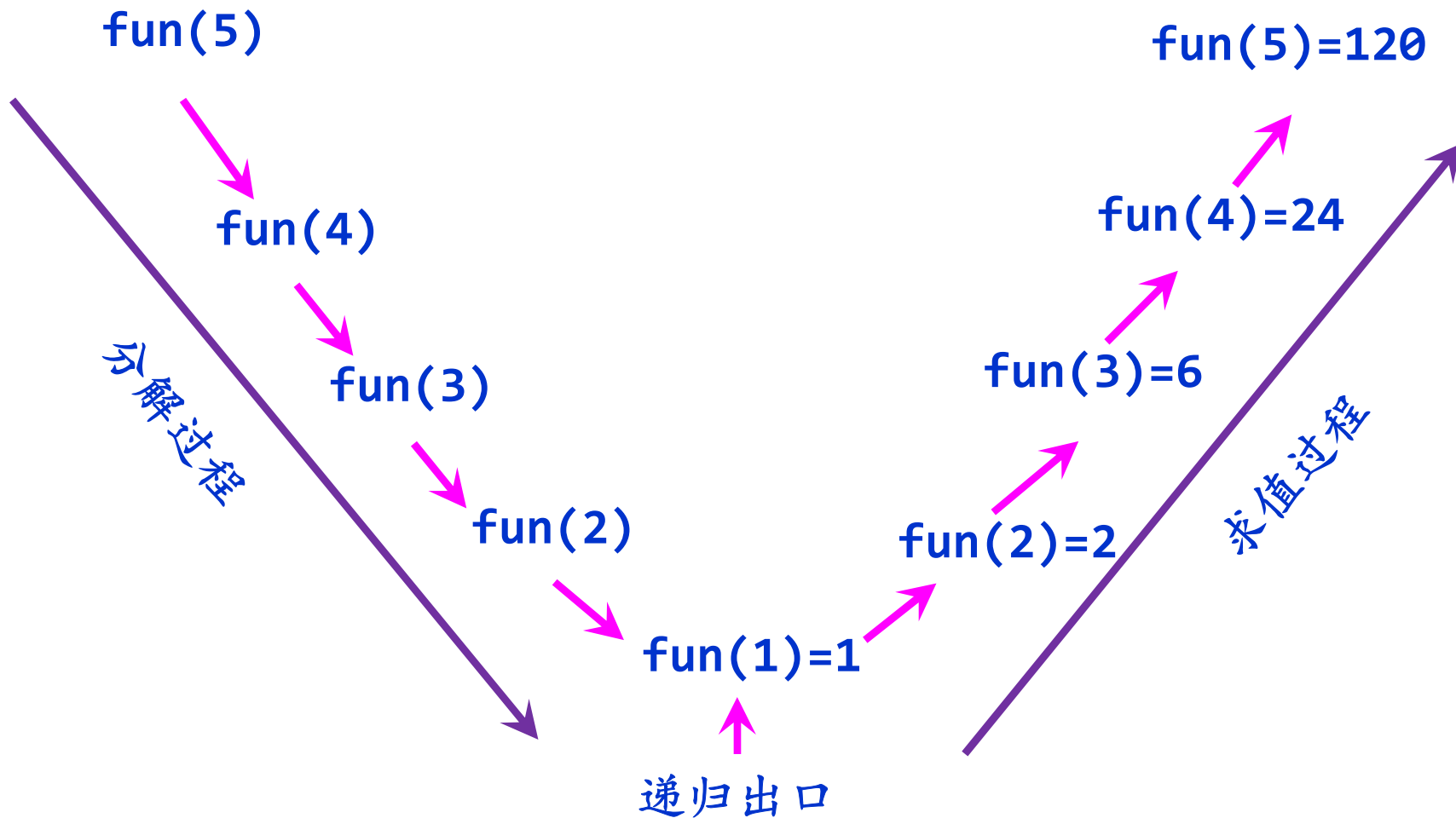
遇到递归出口发生“质变”，即原递归问题便转化成可以直接求解的问题。求值过程：



这样 $f(s_n)$ 便计算出来了，因此递归的执行过程由分解和求值两部分构成。 $f(s_n)=g(f(s_{n-1}),c_{n-1})$



求解 $\text{fun}(5)$ 即 $5!$ 的过程如下:



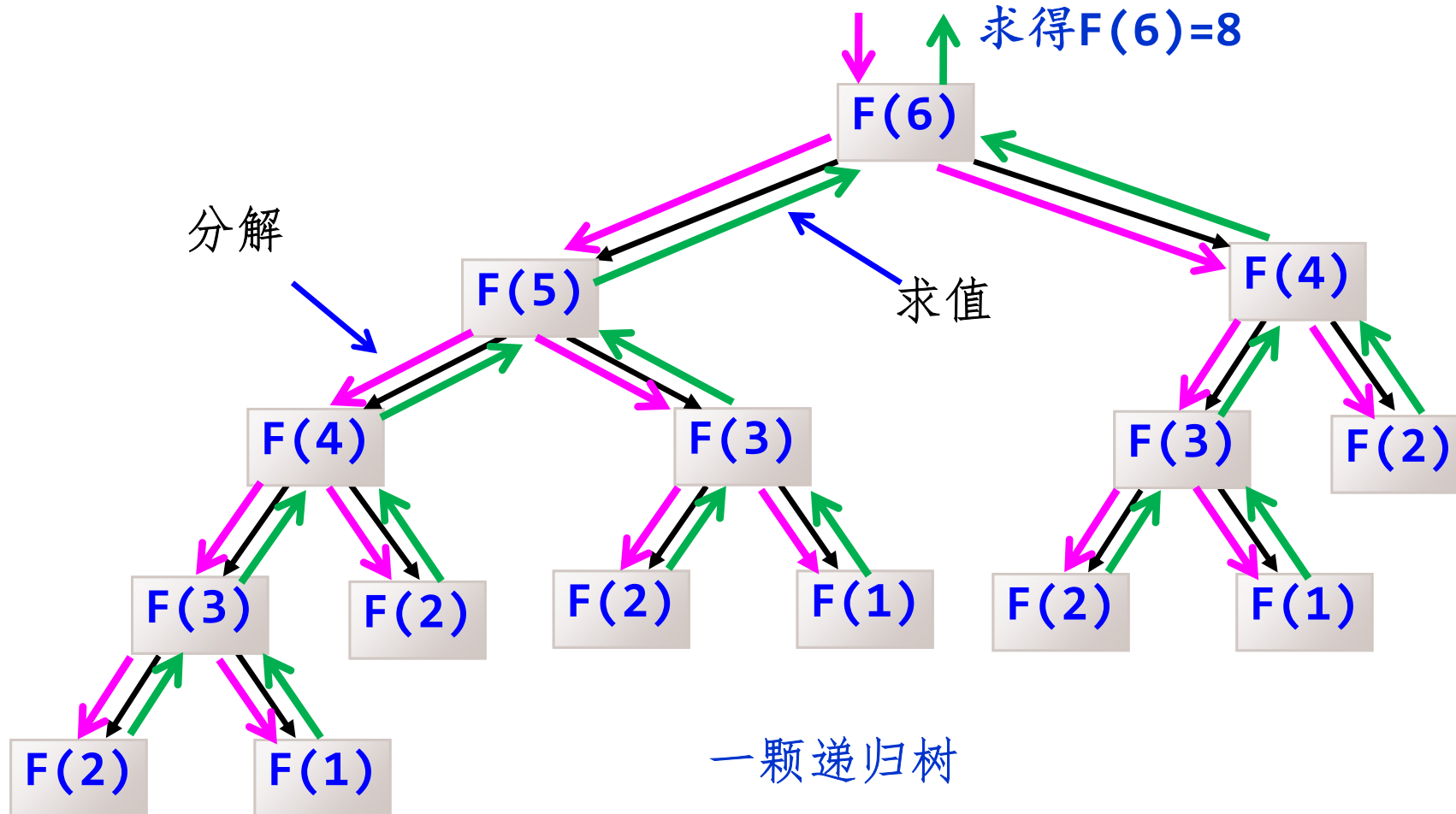


对于复杂的递归问题，在求解时需要进行**多次分解和求值**。

例如： $F(1)=1, F(2)=1$
 $F(n)=F(n-1)+F(n-2) \quad n>2$

求 $F(6) = ?$

求得 $F(6)=8$





5.1.4 递归与数学归纳法

- 从**递归体**看到，如果已知 s_i 、 s_{i+1} 、...、 s_{n-1} ，就可以确定 s_n 。
- 从数学归纳法的角度来看，这相当于数学归纳法归纳步骤的内容。
- 还应给出这个数列的初始值 s_1 ，对应**递归出口**。



例如，采用数学归纳法证明下式：

$$1+2+\cdots+n = n(n+1)/2$$

- ① 当 $n=1$ 时，左式=1，右式= $1*(1+1)/2=1$ ，左右两式相等，等式成立。
- ② 假设当 $n=k-1$ 时等式成立，有 $1+2+\cdots+(k-1)=k(k-1)/2$ 。
- ③ 当 $n=k$ 时，左式= $1+2+\cdots+k=1+2+\cdots+(k-1)+k=k(k-1)/2+k=k(k+1)/2$
等式成立。即证。



利用数学归纳法证明命题分为两个步骤:

- ① 证明当 n 取第一个值 n_0 ($n_0=0$ 或者 1 等) 时结论成立。
- ② 假设 $n=k-1$ (k 属于可数集并且 $k>n_0$) 时结论成立, 证明 $n=k$ 时结论也成立。



结 论

- 数学归纳法是一种论证方法，而递归是算法和程序设计的一种实现技术。
- 数学归纳法是递归求解问题的理论基础。可以说递归的思想来自数学归纳法。



5.2 递归和栈

5.2.1 函数调用栈

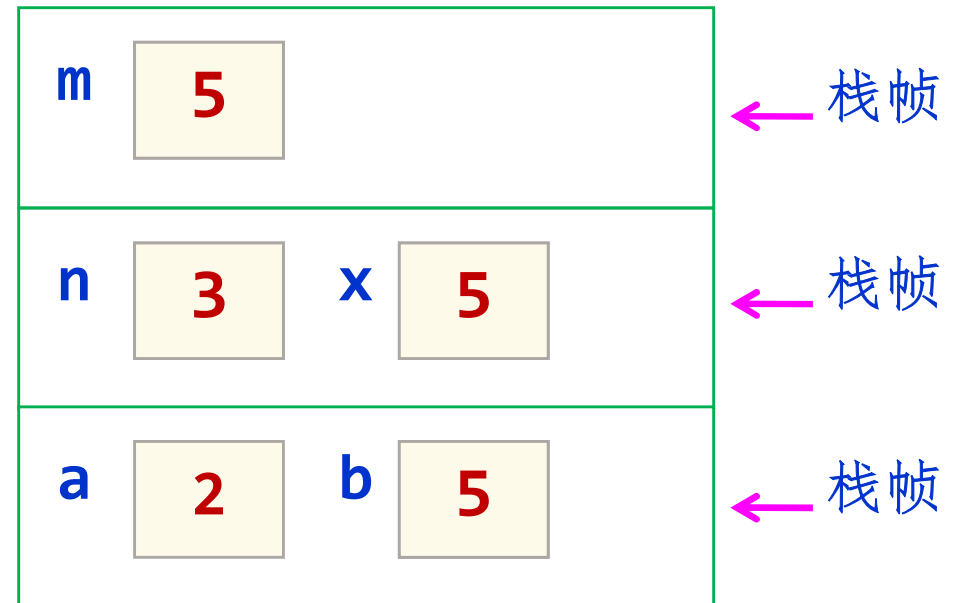
- 函数执行是通过系统栈实现的。系统栈分为若干个栈帧。
- 一次函数调用相关的数据保存在栈帧中。体现先进后出的特点！



例如：

```
int fun1(int n)
{
    int x;
    n++;
    x=fun2(n);
    return x;
}
int fun2(int m)
{
    m+=2;
    return m;
}
void main()
{
    int a=2, b;
    b=fun1(a);
    printf("b=%d\n", b);
}
```

执行过程：



↓

屏幕输出 **b=5**

程序执行完毕



5.2.2 递归函数的实现

- 递归是函数调用的一种特殊情况，即它是调用自身代码。
- 可以把每一次递归调用理解成调用自身代码的一个复制件。由于每次调用时，它的参数和局部变量可能不相同，因而也就保证了各个复制件执行时的独立性。

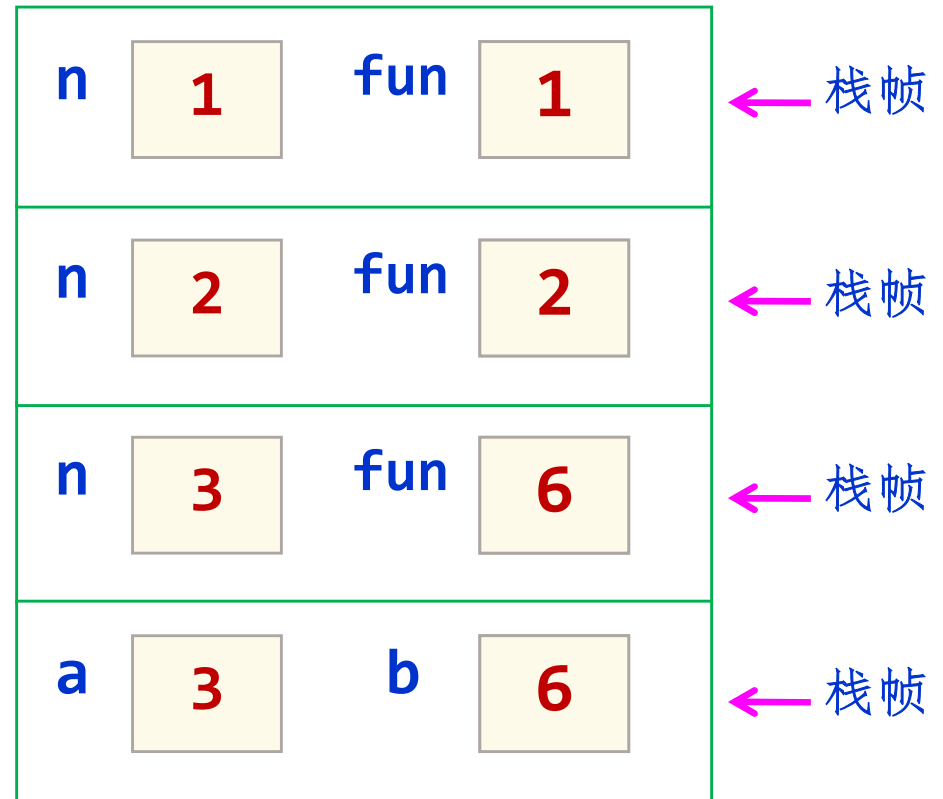


例如：

```
int fun(int n)
{ if (n==1)
    return 1;
  else;
    return fun(n-1)*n;
}

void main()
{ int a=3, b;
  b=fun(a);
  printf("b=%d\n", b);
}
```

执行过程：



屏幕输出 **b=6**
程序执行完毕



5.2.3 递归算法的时空分析

1. 递归算法的时间复杂度分析

递归算法是指算法中出现调用自己的成分。

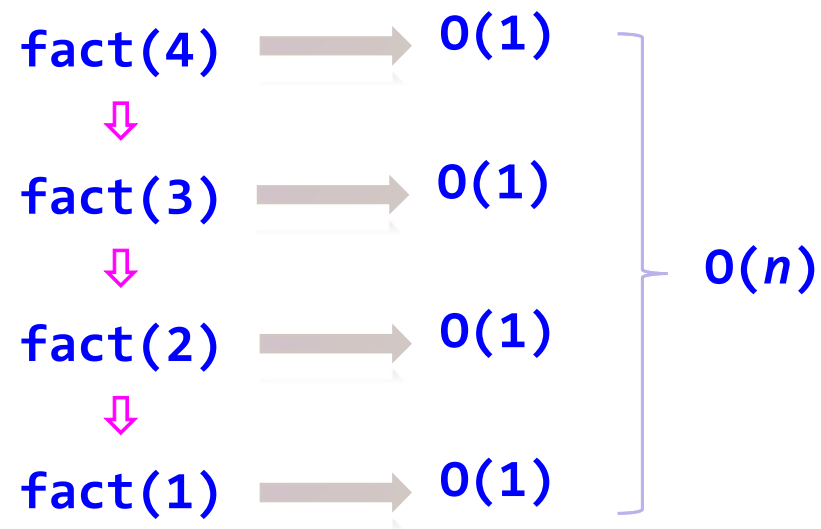
- 递归算法分析也称为变长时空分析。
- 非递归算法分析也称为定长时空分析。



例如，求 $n!$ 的递归算法：

```
int fact(int n)
{  if (n==1)  return 1;
   else return n*fact(n-1);
}
```

求 $\text{fact}(4)$, $n=4$





示例

```
void Hanoi1(int n,char X,char Y,char Z)
{  if (n==1)                                //只有一个盘片的情况
    printf("\t将第%d个盘片从%c移动到%c\n",n,X,Z);
    else                                    //有两个或多个盘片的情况
    {  Hanoi1(n-1,X,Z,Y);
        printf("\t将第%d个盘片从%c移动到%c\n",n,X,Z);
        Hanoi1(n-1,Y,X,Z);
    }
}
```

求Hanoi1(n , x , y , z)的时间复杂度。



```
void Hanoi1(int n,char X,char Y,char Z)
{  if (n==1)                      //只有一个盘片的情况
    printf("\t将第%d个盘片从%c移动到%c\n",n,X,Z);
    else                          //有两个或多个盘片的情况
    {  Hanoi1(n-1,X,Z,Y);
        printf("\t将第%d个盘片从%c移动到%c\n",n,X,Z);
        Hanoi1(n-1,Y,X,Z);
    }
}
```

- 设Hanoi1(n , x , y , z)的执行时间为 $T(n)$ 。
- 则两个问题规模为 $n-1$ 的子问题的执行时间均为 $T(n-1)$ 。
- 总执行时间是累加关系。
- 递推式如下。

$$T(n)=1$$

当 $n=1$ 时

$$T(n)=2T(n-1)+1$$

当 $n>1$ 时



$$T(n)=1$$

当 $n=1$ 时

$$T(n)=2T(n-1)+1$$

当 $n>1$ 时

$$\begin{aligned} T(n) &= 2T(n-1)+1 = 2(2T(n-2)+1)+1 \\ &= 2^2T(n-2)+2+1 = 2^2(2T(n-3)+1)+2+1 \\ &= 2^3T(n-3)+2^2+2+1 \\ &= \dots \\ &= 2^{n-1}T(1)+2^{n-2}+ \dots +2^2+2+1 \\ &= 2^n-1 \\ &= O(2^n)。 \end{aligned}$$



【例5.2】有如下递归算法：

```
void fun(int a[], int n, int k)           //数组a共有n个元素
{
    int i;
    if (k==n-1)
        for (i=0;i<n;i++)
            printf("%d\n", a[i]);         //执行n次
    else
    {
        for (i=k;i<n;i++)
            a[i]=a[i]+i*i;                //执行n-k次
        fun(a, n, k+1);
    }
}
```

调用上述算法的语句为 $\text{fun}(a, n, 0)$ ，求其时间复杂度。



递归算法:

```
void fun(int a[], int n, int k) //数组a共有n个元素
{
    int i;
    if (k==n-1)
        for (i=0;i<n;i++)
            printf("%d\n", a[i]); //执行n次
    else
    {
        for (i=k;i<n;i++)
            a[i]=a[i]+i*i; //执行n-k次
        fun(a, n, k+1);
    }
}
```



含一重循环

$\text{fun}(a, n, 0)$ 的时间复杂度为 $O(n)$ 。

错误



解

设 $\text{fun}(a, n, k)$ 的执行时间为 $T_1(n, k)$ 。

```
void fun(int a[], int n, int k)
```

```
{ int i;
```

```
  if (k==n-1)
```

```
    for (i=0; i<n; i++)
```

```
      printf("%d\n", a[i]);
```

//执行n次

当 $k=n-1$ 时

$$T_1(n, k) = n$$

```
  else
```

```
    { for (i=k; i<n; i++)
```

```
      a[i]=a[i]+i*i;
```

//执行n-k次

当 $k < n-1$ 时

$$T_1(n, k) = (n-k) + T_1(n, k+1)$$

```
    fun(a, n, k+1);
```

```
  }
```

```
}
```

$T_1(n, k)$ 的递推式:

$$T_1(n, k) = n$$

当 $k=n-1$ 时

$$T_1(n, k) = (n-k) + T_1(n, k+1)$$

其他情况



$$T_1(n, k) = n$$

当 $k=n-1$ 时

$$T_1(n, k) = (n-k) + T_1(n, k+1)$$

其他情况

- 设 $\text{fun}(a, n, \theta)$ 的执行时间为 $T(n)$
- 而 $\text{fun}(a, n, k)$ 的执行时间为 $T_1(n, k)$

$$\Rightarrow T(n) = T_1(n, \theta)$$

$$\begin{aligned} T(n) &= T_1(n, \theta) = n + T_1(n, 1) = n + (n-1) + T_1(n, 2) \\ &= \cdots = n + (n-1) + \cdots + 2 + T_1(n, n-1) \\ &= n + (n-1) + \cdots + 2 + n \\ &= n(n+1)/2 + n - 1 \\ &= O(n^2) \end{aligned}$$

所以调用 $\text{fun}(a, n, \theta)$ 的时间复杂度为 $O(n^2)$ 。



设有算法如下 (a 数组元素有序的) :

```
int Find(int a[],int s,int t,int x)
{  int m=(s+t)/2;
   if (s<=t)
   {  if (a[m]==x)
       return m;
      else if (x<a[m])
          return Find(a,s,m-1,x);
      else
          return Find(a,m+1,t,x);
   }
   return -1;
}
```

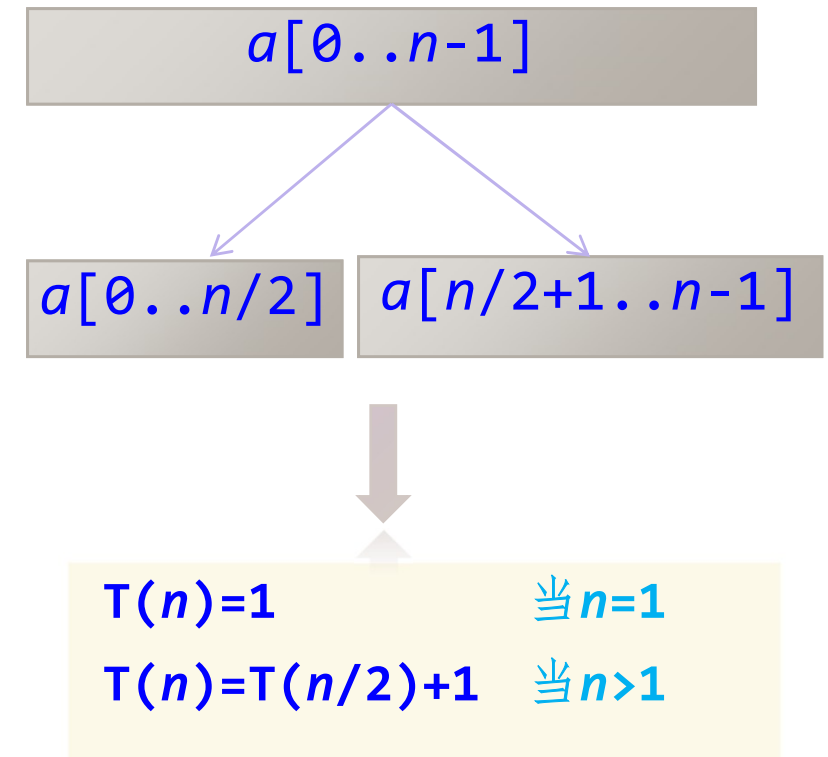
分析Find($a,0,n-1,x$)的时间复杂度是多少。



解

设 $\text{Find}(a, 0, n-1, x)$ 的执行时间为 $T(n)$ 。

```
int Find(int a[], int s, int t, int x)
{
    int m = (s+t)/2;
    if (s <= t)
    {
        if (a[m] == x)
            return m;
        else if (x < a[m])
            return Find(a, s, m-1, x);
        else
            return Find(a, m+1, t, x);
    }
    return -1;
}
```





$$T(n)=1 \quad \text{当 } n=1$$

$$T(n)=T(n/2)+1 \quad \text{当 } n>1$$

不妨设 $n=2^k$, 即 $k=\log_2 n$ 。

$$\begin{aligned} T(n) &= T(n/2)+1 \\ &= T(n/2^2)+2=T(n/2^3)+3 \\ &= \dots \\ &= T(n/2^k)+k \\ &= 1+k \\ &= \log_2 n+1 \\ &= O(\log_2 n)。 \end{aligned}$$

Find(a,0,n-1,x)的时间复杂度是 $O(\log_2 n)$ 。



2. 递归算法的空间复杂度分析

递归算法是指算法中出现调用自己的成分。

- 递归算法分析也称为变长时空分析。
- 非递归算法分析也称为定长时空分析。



示例

```
void Hanoi1(int n,char X,char Y,char Z)
{  if (n==1)                                //只有一个盘片的情况
    printf("\t将第%d个盘片从%c移动到%c\n",n,X,Z);
    else                                    //有两个或多个盘片的情况
    {  Hanoi1(n-1,X,Z,Y);
        printf("\t将第%d个盘片从%c移动到%c\n",n,X,Z);
        Hanoi1(n-1,Y,X,Z);
    }
}
```

求Hanoi1(n , x , y , z)的空间复杂度。



```
void Hanoi1(int n,char X,char Y,char Z)
{  if (n==1)                      //只有一个盘片的情况
    printf("\t将第%d个盘片从%c移动到%c\n",n,X,Z);
    else                          //有两个或多个盘片的情况
    {  Hanoi1(n-1,X,Z,Y);
        printf("\t将第%d个盘片从%c移动到%c\n",n,X,Z);
        Hanoi1(n-1,Y,X,Z);
    }
}
```

- 设Hanoi1(n , x , y , z)的临时空间为 $S(n)$ 。
- 则两个问题规模为 $n-1$ 的子问题的临时空间均为 $S(n-1)$ 。
- 总空间是**最大值**关系。
- 递推式如下。

$$S(n)=1$$

当 $n=1$ 时

$$S(n)=S(n-1)+1$$

当 $n>1$ 时



$$S(n)=1$$

当 $n=1$ 时

$$S(n)=S(n-1)+1$$

当 $n>1$ 时

$$\begin{aligned} S(n) &= S(n-1)+1 \\ &= (S(n-2)+1)+1 \\ &= \cdots \\ &= S(1) + 1 + \cdots + 1 \\ &= 1 + 1 + \cdots + 1 \\ &= \overbrace{0(n)}^{n \uparrow 1} \end{aligned}$$



【例5.3】有如下递归算法，分析调用 $\text{fun}(a, n, 0)$ 的空间复杂度。

```
void fun(int a[], int n, int k) //数组a共有n个元素
{
    int i;
    if (k==n-1)
        for (i=0;i<n;i++)
            printf("%d\n", a[i]); //执行n次
    else
    {
        for (i=k;i<n;i++)
            a[i]=a[i]+i*i; //执行n-k次
        fun(a, n, k+1);
    }
}
```



递归算法:

```
void fun(int a[], int n, int k) //数组a共有n个元素
{
    int i;
    if (k==n-1)
        for (i=0;i<n;i++)
            printf("%d\n", a[i]); //执行n次
    else
    {
        for (i=k;i<n;i++)
            a[i]=a[i]+i*i; //执行n-k次
        fun(a, n, k+1);
    }
}
```

仅仅定义了一个临时变量*i*

$\text{fun}(a, n, 0)$ 的空间复杂度为 $O(1)$ 。

错误



解

设 $\text{fun}(a, n, k)$ 的空间为 $S_1(n, k)$

```
void fun(int a[], int n, int k)
```

```
{  int i;
```

```
    if (k==n-1)
```

```
        for (i=0;i<n;i++)
```

```
            printf("%d\n", a[i]); //执行n次
```

```
    else
```

```
    {  for (i=k;i<n;i++)
```

```
        a[i]=a[i]+i*i; //执行n-k次
```

```
        fun(a, n, k+1);
```

```
    }
```

```
}
```

当 $k=n-1$ 时

$$S_1(n, k) = 1$$

当 $k < n-1$ 时

$$S_1(n, k) = 1 + S_1(n, k+1)$$

$S_1(n, k)$ 的递推式

$$S_1(n, k) = 1$$

当 $k=n-1$ 时

$$S_1(n, k) = 1 + S_1(n, k+1)$$

其他情况



- 设 $\text{fun}(a, n, \theta)$ 的空间为 $S(n)$
- 而 $\text{fun}(a, n, k)$ 的空间为 $S_1(n, k)$ $\Rightarrow S(n) = S_1(n, \theta)$

$$S_1(n, k) = 1 \quad \text{当 } k=n-1 \text{ 时}$$

$$S_1(n, k) = 1 + S_1(n, k+1) \quad \text{其他情况}$$

则：

$$\begin{aligned} S(n) &= S_1(n, \theta) = 1 + S_1(n, 1) = 1 + 1 + S_1(n, 2) \\ &= \dots = 1 + 1 + \dots + 1 = O(n) \end{aligned}$$

$\underbrace{\hspace{10em}}_{n \text{ 个 } 1}$

所以调用 $\text{fun}(a, n, \theta)$ 的空间复杂度为 $O(n)$ 。



5.2.4 递归到非递归的转换

1

直接转换法

- **直接转换法**就是用迭代方式或者循环语言替代多次重复的递归调用。
- **直接转换法**通常用来消除尾递归和单向递归。
- 如果一个递归函数中递归调用语句是最后一条执行语句且把当前运算结果放在参数里传给下层函数，称这种递归调用称为**尾递归**。



```
int fact(int n)
{  if (n==1)           //语句1
    return 1;          //语句2
    else                //语句3
    return n*fact(n-1); //语句4
}
```



- 递归调用语句是最后一条执行语句
- 把当前运算结果放在参数里传给下层函数

```
int fact1(int n,int ans) //求n!的尾递归算法
{  if(n==1)
    return ans;
    else
    return fact1(n-1,n*ans);
}
```



- **单向递归**是指递归的求值过程总是朝着一个方向进行的。

```
int Fib1(int n)
{
    if (n==1 || n==2)
        return(1);
    else
        return(Fib1(n-1)+Fib1(n-2));
}
```

求Fibonacci数列:

1 1 2 3 5 8 ...



```
int Fib2(int n)
{
    int a=1, b=1, i, s;
    if (n==1 || n==2)
        return(1);
    else
    {
        for (i=3; i<=n; i++)
        {
            s=a+b;
            a=b;
            b=s;
        }
        return s;
    }
}
```



2

间接转换法

- 其他相对复杂的递归算法不能直接求值，执行中需要回溯。
- 可以在理解递归调用实现过程的基础上，用栈来模拟递归执行过程即使用栈保存中间结果，从而将其转换为等效的非递归算法。称为间接转换法。



Hanoi问题求解递归算法

```
void Hanoi1(int n, char X, char Y, char Z)
{
    if (n==1)                //只有一个盘片的情况
        printf("\t将第%d个盘片从%c移动到%c\n", n, X, Z);
    else                      //有两个或多个盘片的情况
    {
        Hanoi1(n-1, X, Z, Y);
        printf("\t将第%d个盘片从%c移动到%c\n", n, X, Z);
        Hanoi1(n-1, Y, X, Z);
    }
}
```



Hanoi问题求解非递归算法

设计顺序栈的类型如下

```
typedef struct
{
    int n;                // 盘片个数
    char x, y, z;         // 3个塔座
    bool flag;             // 可直接移动盘片时为true, 否则为false
} ElemType;              // 顺序栈中元素类型

typedef struct
{
    ElemType data[MaxSize]; // 存放元素
    int top;                // 栈顶指针
} StackType;              // 顺序栈类型
```




```
void Hanoi2(int n, char x, char y, char z)
```

```
{
```

```
    StackType *st;
```

```
//定义顺序栈指针
```

```
    ElemType e, e1, e2, e3;
```

```
    if (n<=0) return;
```

```
//参数错误时直接返回
```

```
    InitStack(st);
```

```
//初始化栈
```

```
    e.n=n; e.x=x; e.y=y; e.z=z; e.flag=false;
```

```
    Push(st, e);
```

```
//元素e进栈
```

Hanoi(n, X, Y, Z)任务进栈



```
while (!StackEmpty(st))           //栈不空循环
{
    Pop(st, e);                   //出栈元素e
    if (e.flag==false)            //当不能直接移动盘片时
    {
        e1.n=e.n-1; e1.x=e.y; e1.y=e.x; e1.z=e.z;
        if (e1.n==1)              //只有一个盘片时可直接移动
            e1.flag=true;
        else                      //有一个以上盘片时不能直接移动
            e1.flag=false;
        Push(st, e1);             //处理Hanoi(n-1, y, x, z)步骤
    }
}
```

Hanoi(n, X, Y, Z)任务

- ① **Hanoi**(n-1, X, Z, Y)任务
- ② **Move**(n, X, Z): flag=true
- ③ **Hanoi**(n-1, Y, X, Z)任务



```
e2.n=e.n; e2.x=e.x; e2.y=e.y; e2.z=e.z; e2.flag=true;
Push(st, e2); //处理move(n, x, z)步骤
e3.n=e.n-1; e3.x=e.x; e3.y=e.z; e3.z=e.y;
if (e3.n==1) //只有一个盘片时可直接移动
    e3.flag=true;
else
    e3.flag=false; //有一个以上盘片时不能直接移动
Push(st, e3); //处理Hanoi(n-1, x, z, y)步骤
}
```

① Hanoi(n-1, X, Z, Y)任务进栈

Hanoi(n, X, Y, Z)任务

② Move(n, X, Z): flag=true

③ Hanoi(n-1, Y, Z, X)任务进栈



else

//当可以直接移动时

printf("\t将第%d个盘片从%c移动到%c\n", e.n, e.x, e.z);

}

DestroyStack(st);

//销毁栈

}

求解Hanoi(n, X, Y, Z): flag=true



5.3 递归算法的设计

5.3.1 递归算法设计的步骤

- 设计求解问题的递归模型。
- 转换成对应的递归算法。

递归模型



递归算法



求递归模型的步骤如下：

(1) 对原问题 $f(s)$ 进行分析，称为“大问题”，假设出合理的“小问题” $f(s')$ ；

(2) 假设 $f(s')$ 是可解的，在此基础上确定 $f(s)$ 的解，即给出 $f(s)$ 与 $f(s')$ 之间的关系 $\Rightarrow$ **递归体**。

(3) 确定一个特定情况（如 $f(1)$ 或 $f(0)$ ）的解 $\Rightarrow$ **递归出口**。

数学归纳法

假设 $n=k-1$ 时等式成立
求证 $n=k$ 时等式成立

求证 $n=1$ 时等式成立

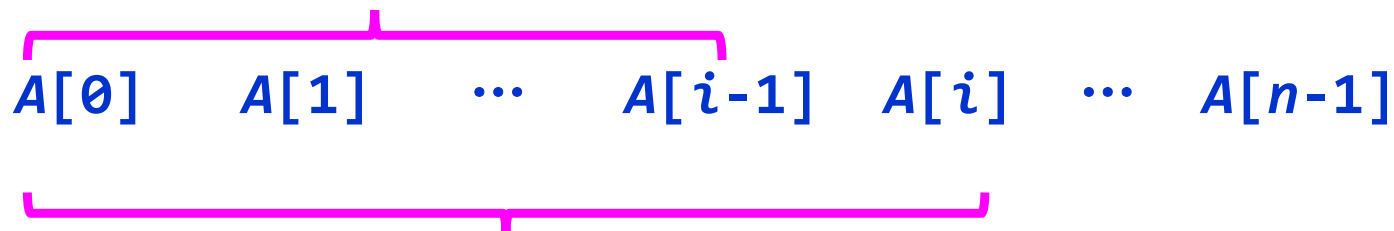


【例5.4】采用递归算法求实数数组 $A[0..n-1]$ 中的最小值。

解

假设 $f(A, i)$ 求数组元素 $A[0] \sim A[i]$ ($i+1$ 个元素) 中的最小值。

$f(A, i-1)$: 小问题, 处理 i 个元素



$f(A, i)$: 大问题, 处理 $i+1$ 个元素

假设 $f(A, i-1)$ 已求出, 则 $f(A, i) = \text{MIN}(f(A, i-1), A[i])$, 其中 $\text{MIN}()$ 为求两个值较小值函数。

当 $i=0$ 时, 只有一个元素, 有 $f(A, i) = A[0]$ 。

因此得到如下递归模型:

$$f(A, i) = A[0]$$

当 $i=0$ 时

$$f(A, i) = \text{MIN}(f(A, i-1), A[i])$$

其他情况



由此得到如下递归求解算法：

```
double Min(double A[], int i)
{
    double min;
    if (i==0)
        return A[0];
    else
    {
        min=Min(A, i-1);
        if (min>A[i])
            return A[i];
        else
            return min;
    }
}
```

调用方式：

← `double a[]={1,2,3};`
`int n=3;`
`double ans=f(a,n-1);`



【例5.5】求含 n ($n>1$) 个元素的顺序表 L 中的最大元素。

解法1

- 顺序表 L 采用数组 $\text{data}[0..n-1]$ 存放全部元素。
- 采用与例5.4类似的思路。

```
ElemType Max1(SqList L,int i)  //解法1: 求顺序表L中最大元素
{
    ElemType max;
    if (i==0)
        return L.data[0];
    else
    {
        max=Max1(L,i-1);
        if (max<L.data[i])
            return L.data[i];
        else
            return max;
    }
}
```



解法2

- 假设顺序表L中data数组的元素为 $(a_0, a_1, \dots, a_{n-1})$ 。
- 将其分解成 $(a_0, a_1, \dots, a_m)$ 和 $(a_{m+1}, \dots, a_{n-1})$ 左右两个子表。
- 分别求得它们的最大元素为max1和max2，则整个顺序表L的最大元素就是max1和max2中较大者。

递归模型

$f(a, i, j) = a[i]$

当 $i=j$

$f(a, i, j) = \max$

$\text{mid} = (i+j)/2$

其他情况

$\text{max1} = f(a, i, \text{mid})$

$\text{max2} = f(a, \text{mid}+1, j)$

$\text{max} = \text{MAX}(\text{max1}, \text{max2})$



```
ElemType Max2(SqList L,int i,int j) //解法2: 求顺序表L中最大元素
{
    int mid;
    ElemType max,max1,max2;
    if (i==j) //顺序表中只有一个元素, 即递归出口
        max=L.data[i]; //该元素就是最大元素
    else //顺序表中有多个元素
    {
        mid=(i+j)/2; //求中间位置
        max1=Max2(L,i,mid); //递归调用求左子表最大元素max1
        max2=Max2(L,mid+1,j); //递归调用求右子表最大元素max2
        max=(max1>max2)?max1:max2; //求整个表的最大元素max
    }
    return(max);
}
```

5.3.2 基于递归数据结构的递归算法设计

递归数据结构的数据特别适合递归处理 $\Rightarrow$ 递归算法

种瓜得瓜：递归性



数据: $D = \{\text{瓜的集合}\}$

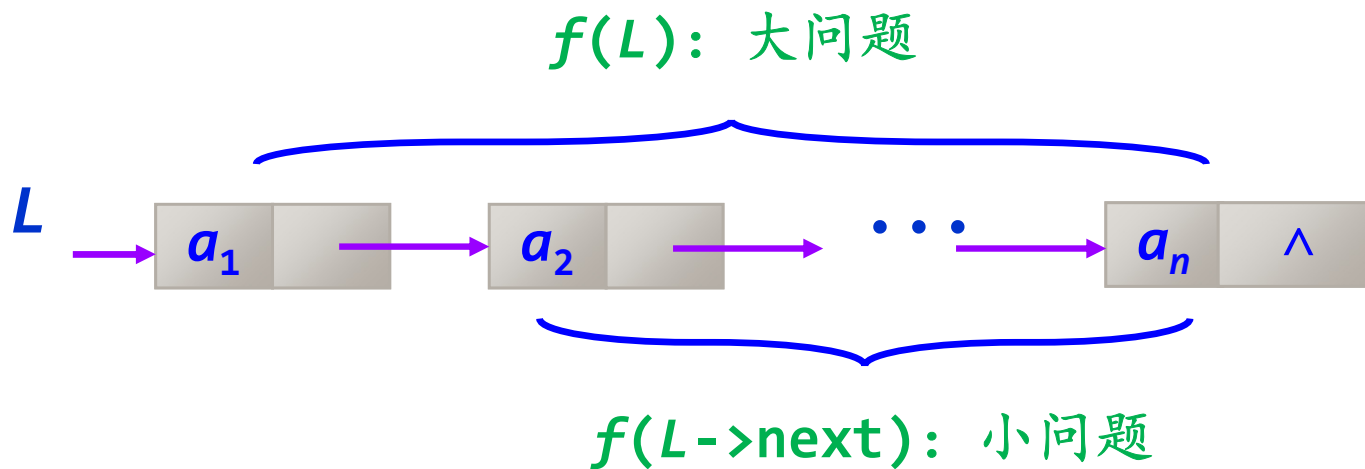
运算: $Op = \{\text{种瓜}\}$

递归性:

$Op(x \in D) \in D$



【例（补充）】设计不带头结点的单链表的相关递归算法。



把“大问题”转化为若干个相似的“小问题”来求解。

为什么在这里设计单链表的递归算法时不带头结点？

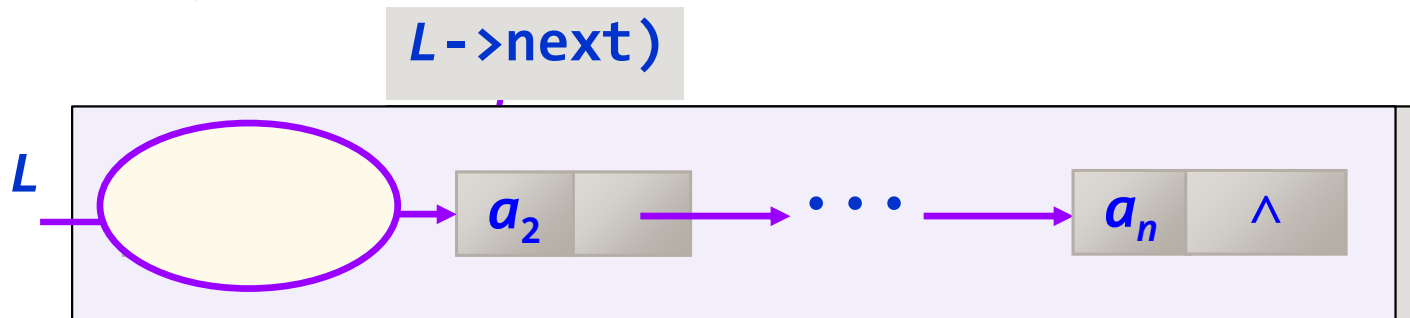


① 求单链表中数据结点个数。

设 $f(L)$ 为单链表中数据结点个数。

☑ 空单链表的数据结点个数为0 $\Rightarrow f(L)=0$ 当 $L=NULL$

☑ 对于非空单链表:



$$f(L) = f(L \rightarrow next) + 1$$

递归模型:

$f(L)=0$	当 $L=NULL$
$f(L)=f(L \rightarrow next)+1$	其他情况



求单链表中数据结点个数递归算法如下：

```
int count(LinkNode *L)
{  if (L==NULL)
    return 0;
    else
    return count(L->next)+1;
}
```



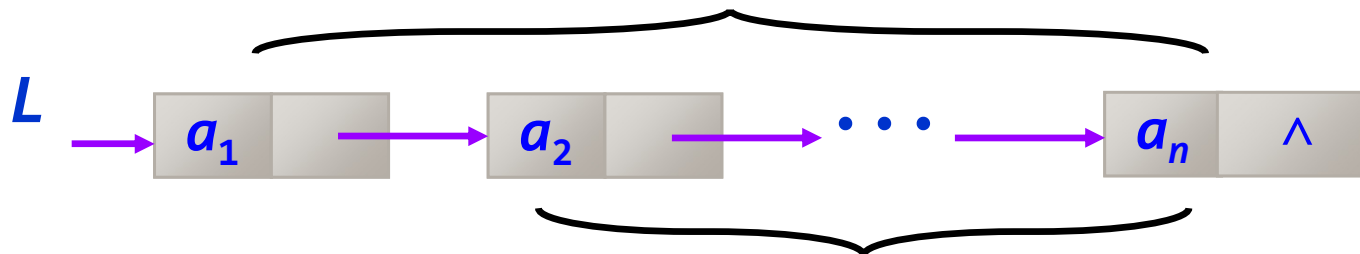
不带头结点单链表L

② 正向显示所有结点值。

③ 反向显示所有结点值。

$f(L)$: 大问题, 输出 $a_1, a_2, \dots, a_n$

$f(L)$: 大问题, 输出 $a_n, \dots, a_2 a_1$



$f(L \rightarrow \text{next})$: 小问题, 输出 $a_n, \dots, a_2$ a_n

假设 $f(L \rightarrow \text{next})$ 已求解

$f(L) \Rightarrow f(L \rightarrow \text{next});$ 输出 $L \rightarrow \text{data};$

$f(L) \Rightarrow$ 输出 $L \rightarrow \text{data};$ $f(L \rightarrow \text{next});$



不带头结点单链表L

正向显示所有结点值。

递归模型：

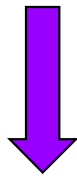
$f(L) \leftrightarrow$ 不做任何事件

当 $L=NULL$

$f(L) \leftrightarrow$ 输出 $L \rightarrow data; f(L \rightarrow next)$

其他情况

递归算法



```
void traverse(LinkNode *L)
{
    if (L==NULL) return;
    printf("%d ", L->data);
    traverse(L->next);
}
```

反向显示所有结点值。

递归模型：

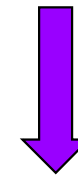
$f(L) \leftrightarrow$ 不做任何事件

当 $L=NULL$

$f(L) \leftrightarrow f(L \rightarrow next);$ 输出 $L \rightarrow data$

其他情况

递归算法



```
void traverseR(LinkNode *L)
{
    if (L==NULL) return;
    traverseR(L->next);
    printf("%d ", L->data);
}
```

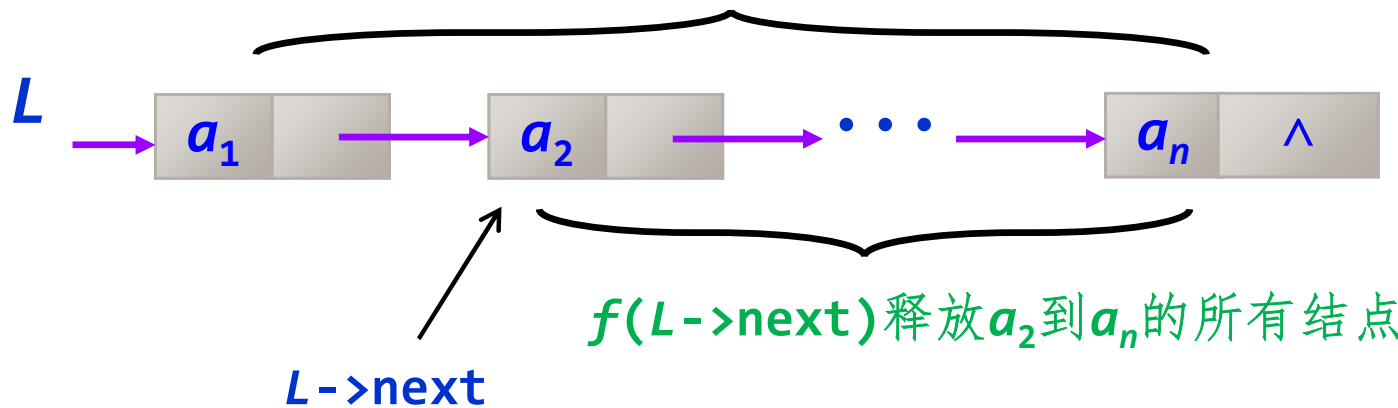


【例5.6】假设有一个不带头结点的单链表 L ，设计一个算法释放其中所有结点。

解

设 $f(L)$ 的功能是释放 $a_1 \sim a_n$ 的所有结点。

$f(L)$ 释放 a_1 到 a_n 的所有结点



递归模型：

$f(L) \equiv$ 不做任何事件

当 $L = \text{NULL}$ 时

$f(L) \equiv f(L \rightarrow \text{next})$; 释放 L 所指结点

其他情况



$f(L) \equiv$ 不做任何事件

当 $L=NULL$ 时

$f(L) \equiv f(L \rightarrow next)$; 释放 L 所指结点

其他情况



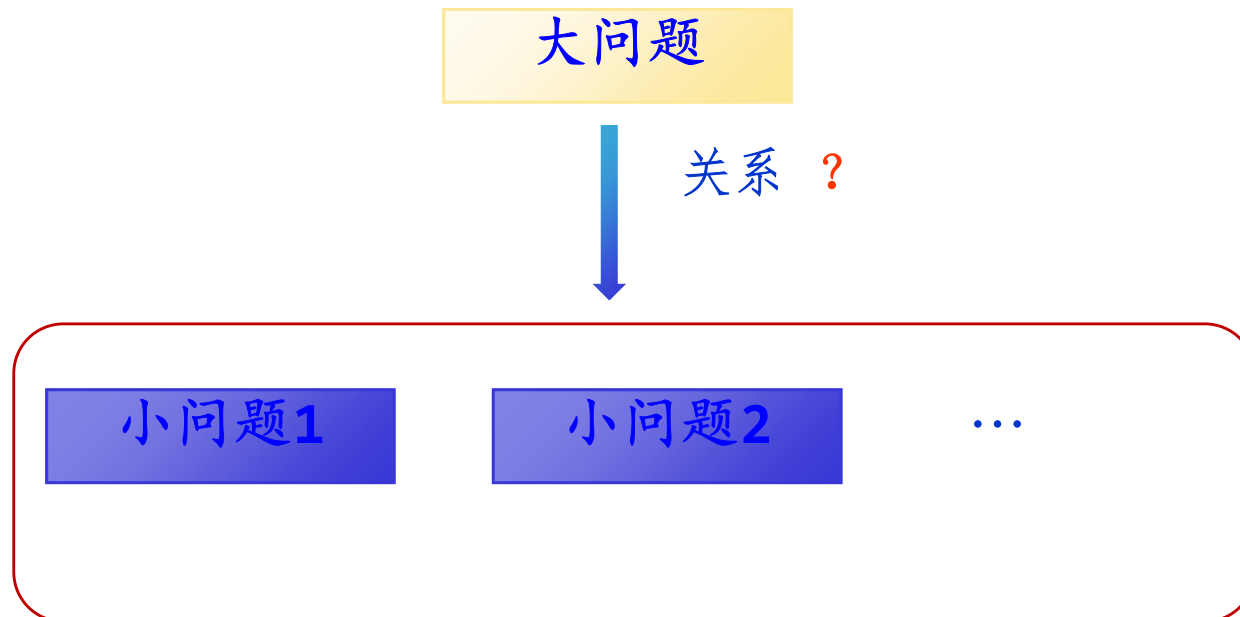
```
void release(LinkNode *&L)
{  if (L!=NULL)
    {  release(L->next);
        free(L);
    }
}
```

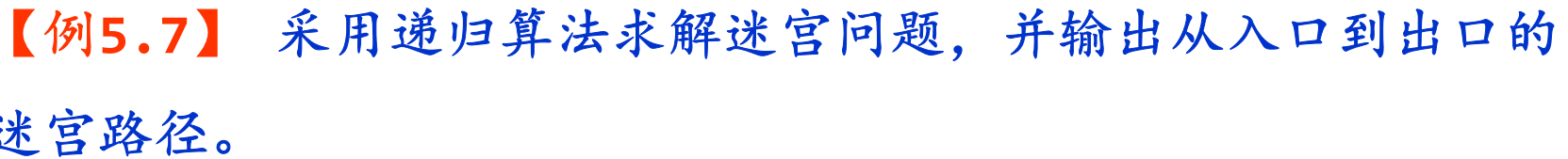


5.3.3 基于递归求解方法的递归算法设计

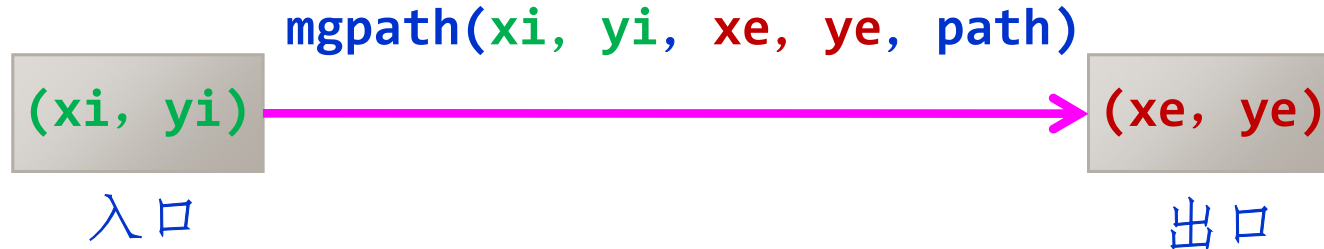
有些问题可以采用递归方法求解（求解方法之一）。

采用递归方法求解问题时，需要对问题本身进行分析，确定大、小问题解之间的关系，构造合理的递归体。



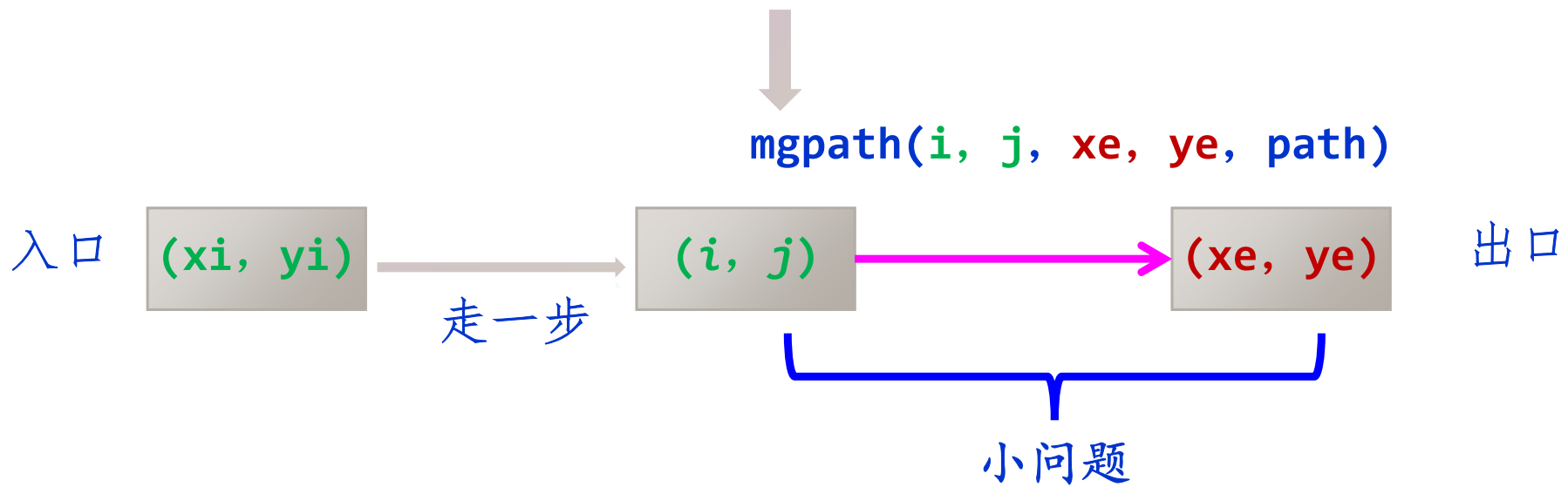
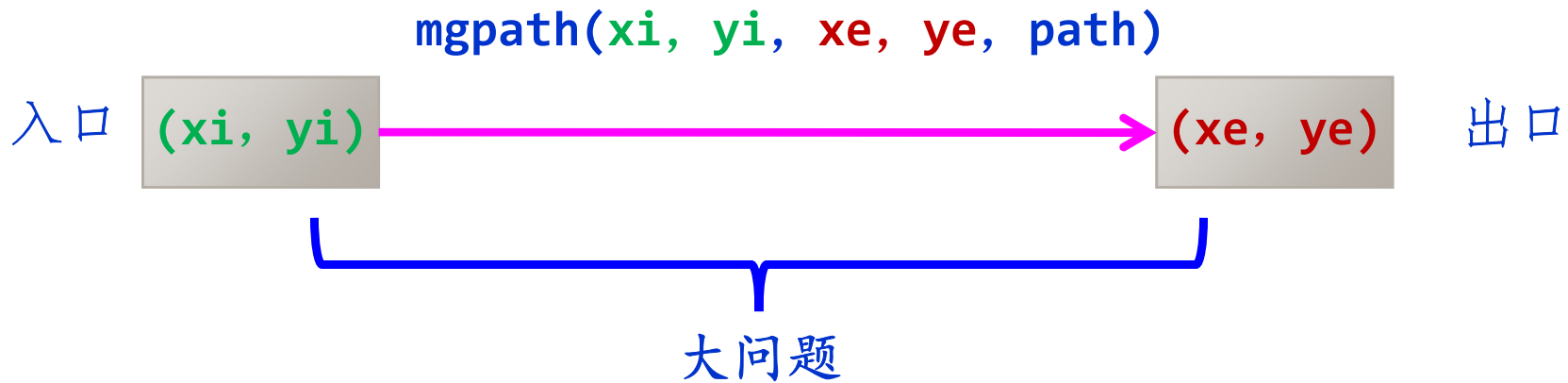


求解问题描述:



```
mopath(int xi, int yi, int xe, int ye, PathType path):
```

求从(xi, yi)到(xe, ye)的迷宫路径，用path变量保存迷宫路径。



大问题 $\equiv$ 走一步 + 小问题



求解迷宫问题的递归模型:

$\text{mgpath}(xi, yi, xe, ye, \text{path}) \equiv$ 将 (xi, yi) 添加到 path 中;

输出 path 中的迷宫路径;

若 $(xi, yi) = (xe, ye)$

$\text{mgpath}(xi, yi, xe, ye, \text{path}) \equiv$ 对于 (xi, yi) 四周的每一个相邻方块 (i, j) :

① 将 (xi, yi) 添加到 path 中;

② 置 $\text{mg}[xi][yi] = -1$;

③ $\text{mgpath}(i, j, xe, ye, \text{path});$

④ path 回退一步并置 $\text{mg}[xi][yi] = 0$;

若 (xi, yi) 不为出口且可走

在一个“小问题”执行完后回退找所有解



迷宫路径用顺序表存储，它的元素由方块构成的。
其PathType类型定义如下：

```
typedef struct
{
    int i;                //当前方块的行号
    int j;                //当前方块的列号
} Box;

typedef struct
{
    Box data[MaxSize];
    int length;           //路径长度
} PathType;              //定义路径类型
```




```
void mgpath(int xi, int yi, int xe, int ye, PathType path)
//求解路径为:(xi, yi)  →  (xe, ye)
{  int di, k, i, j;
   if  (xi==xe && yi==ye)
   {
       path.data[path.length].i = xi;
       path.data[path.length].j = yi;
       path.length++;
       printf("迷宫路径%d如下:\n", ++count);
       for (k=0;k<path.length;k++)
       {   printf("\t(%d, %d)", path.data[k].i,  path.data[k].j);
           if ((k+1)%5==0)           //每输出每5个方块后换一行
               printf("\n");
       }
       printf("\n");
   }
}
```

找到了出口，输出路径（递归出口）



```
else                                     //(xi, yi)不是出口
{   if (mg[xi][yi]==0)                 //(xi, yi)是一个可走方块
    {   di=0;
        while (di<4)                  //对于(xi, yi)四周的每一个相邻方位di
        {   switch(di)                //找方位di对应的方块(i, j)
            {
                case 0:i=xi-1; j=yi;   break;
                case 1:i=xi;   j=yi+1; break;
                case 2:i=xi+1; j=yi;   break;
                case 3:i=xi;   j=yi-1; break;
            }
            ① path.data[path.length].i = xi;
              path.data[path.length].j = yi;
              path.length++;           //路径长度增1
            ② mg[xi][yi]=-1;           //避免来回重复找路径
```

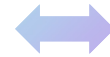


```
③ mgpath(i, j, xe, ye, path);  
④ path.length--;           //回退一个方块  
mg[xi][yi]=0;              //恢复(xi, yi)为可走  
di++;  
    }                       //-while  
}                             //- if (mg[xi][yi]==0)  
}                             //-递归体  
}
```

本算法输出所有的迷宫路径，可以通过进一步比较找出最短路径（可能存在多条最短路径）。

应用

	0	1	2	3	4	5
0						
1		💧				
2						
3						
4					😊	
5						



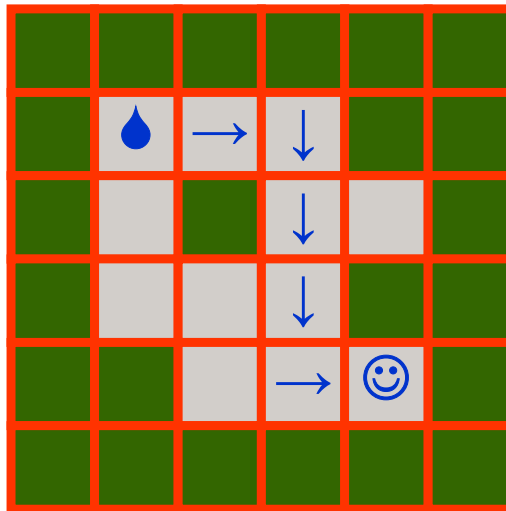
```
int mg[M+2][N+2]=      //M=4, N=4
{  {1,  1,  1,  1,  1,  1},
   {1,  0,  0,  0,  1,  1},
   {1,  0,  1,  0,  0,  1},
   {1,  0,  0,  0,  1,  1},
   {1,  1,  0,  0,  0,  1},
   {1,  1,  1,  1,  1,  1}  };
```

```
void main()
{  PathType path;
   path.length=0;
   mgpath(1, 1, 4, 4, path);
}
```

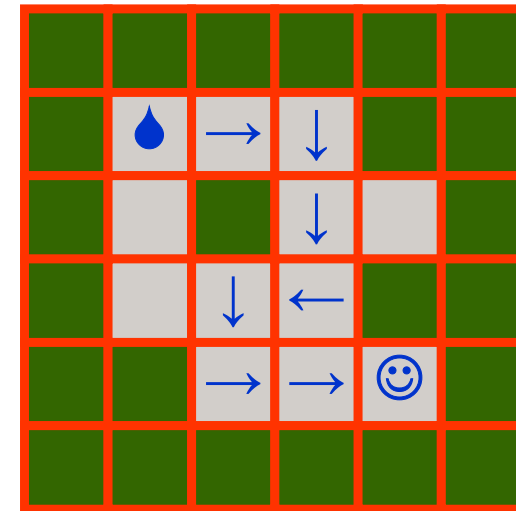


得到如下4条迷宫路径:

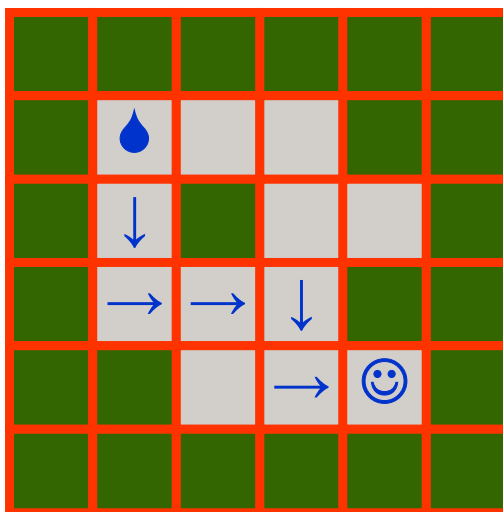
迷宫路径
1



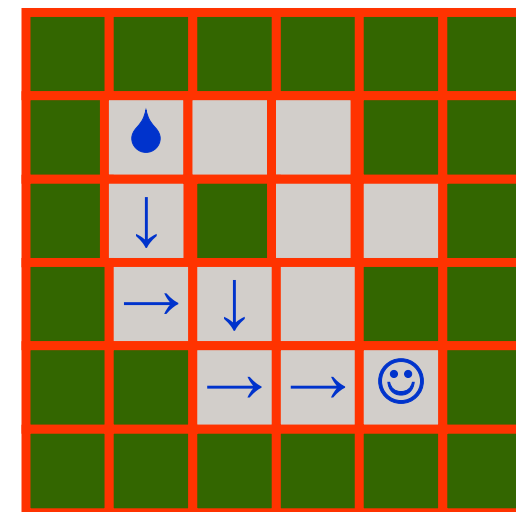
迷宫路径
2



迷宫路径
3



迷宫路径
4





思考题：

迷宫问题的递归求解与用栈和队列求解有什么异同？



第5章小结

1

递归基础

① 一个递归模型由哪两部分构成？



- **递归出口**—确定递归结束情况
- **递归体**—确定大小问题的求解情况



② 递归算法如何转换为非递归算法?

- 对于尾递归和单向递归，可以用循环递推方法来转换。
- 对于其他递归，可以用栈模拟执行过程来转换。



③ 在Hanoi问题的递归算法中，当移动6个盘片时递归次数是多少？

$$m(n) = 1 \quad \text{当 } n=1$$

$$m(n) = 2m(n-1)+1 \quad \text{当 } n>1$$



$$\begin{aligned} t(6) &= 2t(5) + 1 \\ &= 2^2t(4) + 1 + 2 \\ &= 2^3t(3) + 1 + 2 + 2^2 \\ &= 2^4t(2) + 1 + 2 + 2^2 + 2^3 \\ &= 2^5t(1) + 1 + 2 + 2^2 + 2^3 + 2^4 \\ &= 1 + 2 + 2^2 + 2^3 + 2^4 + 2^5 = 2^6 - 1 = 63 \end{aligned}$$



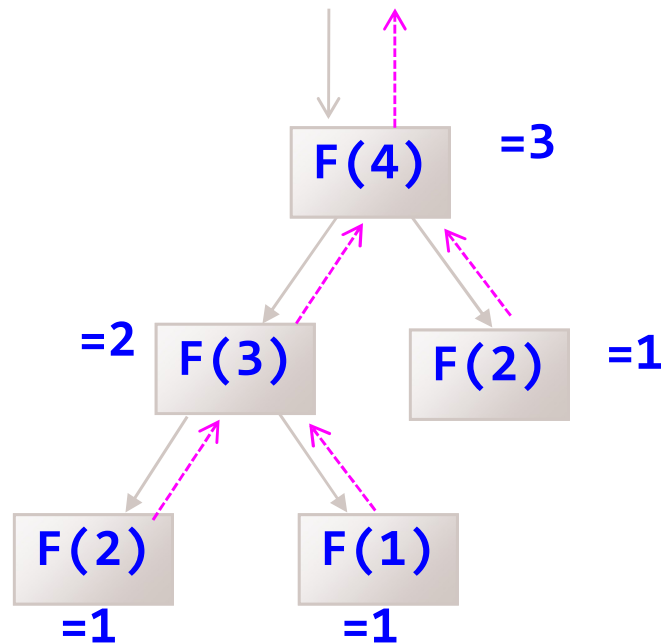
④ 分析递归求Fibonacci数列时，栈的变化情况？

$$F(1)=1$$

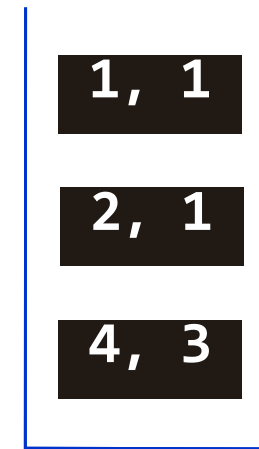
$$F(2)=1$$

$$F(n)=F(n-1)+F(n-2) \quad n>2$$

求 $F(4) = ?$



栈



参数，函数值

求出 $F(4)=3$



2

递归算法设计

① 基于递归数据结构的递归算法设计

- 利用递归数据结构的递归特性建立递归模型
- 编写对应的递归算法



许多单链表的算法均可以采用递归算法实现！



② 基于递归方法的递归算法设计

如何将递归特性不明显的问题转化为递归问题求解



- 确定问题的形式化描述
- 确定哪些是大问题，哪些是小问题
- 确定大、小问题的关系
- 确定特殊（递归结束）情况



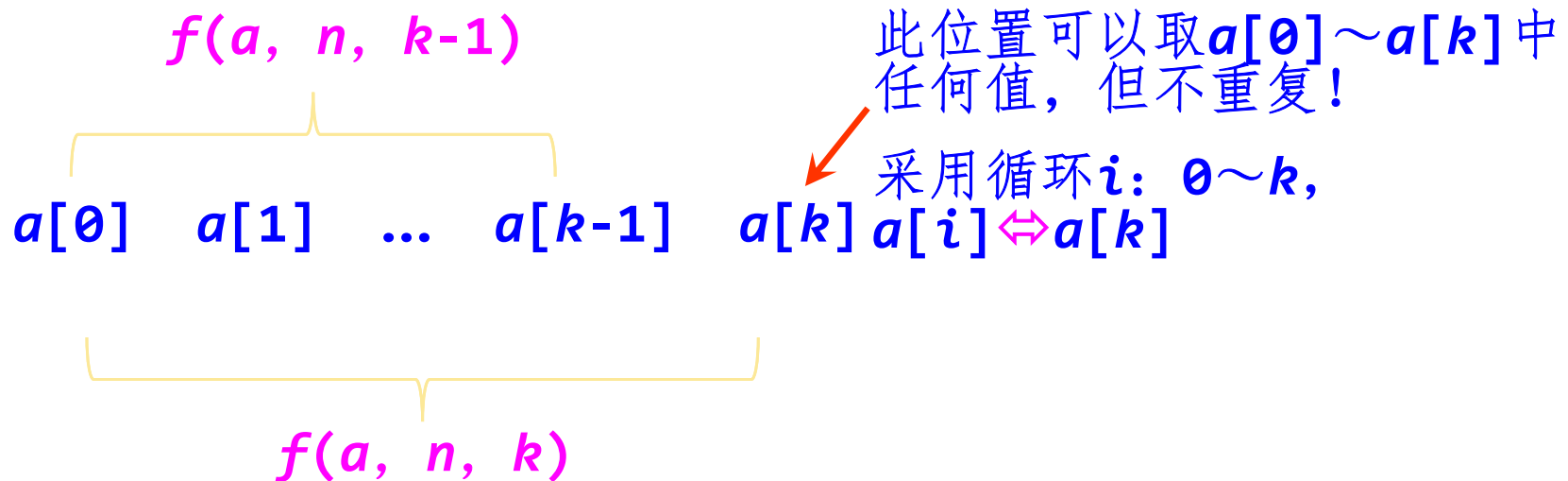
【例】假设 a 数组含有 $1, 2, \dots, n$ ，求其全排列。

解

- 设 $f(a, n, k)$ 为 $a[0..k]$ ($k+1$ 个元素) 的所有元素的全排序，为大问题。
- 则 $f(a, n, k-1)$ 为 $a[0..k-1]$ (k 个元素) 的所有元素的全排序，为小问题。



假设 $f(a, n, k-1)$ 可求, 对于 $a[k]$ 位置, 可以取 $a[0..k]$ 任何元素值, 再组合 $f(a, n, k-1)$, 则得到 $f(a, n, k)$ 。



$f(a, n, k) \Leftrightarrow$ 输出 a 当 $k=0$ 时(一个元素的全排列)

$f(a, n, k) \Leftrightarrow a[k]$ 位置取 $a[0..k]$ 任何之值, 其他情况

并组合 $f(a, n, k-1)$ 的结果;



```
void perm(int a[], int n, int k)
{  int  i, j;
   if  (k==0)
   {  for (j=0;j<n;j++)
       printf("%d", a[j]);
       printf("\n");
   }
   else
   {  for (i=0;i<=k;i++)
       {  swap(a[k], a[i]);
          perm(a, n, k-1);
          swap(a[k], a[i]);
       }
   }
}
```

```
void main()
{  int n=3, k=2;
   int a[]={1,2,3};
   perm(a, n, k);
}
```

输出结果:

231
321
312
132
213
123



3

递归函数设计中几个问题

- ① 递归函数中的引用形参可以用全局变量代替

例如，求 $1 + 2 + \cdots + n$

```
void add(int n, int &s)    //s=1+2+...+n
{
    int s1;
    if (n==1)
        s=1;
    else
    {
        add(n-1, s1);
        s=s1+n;
    }
}
```



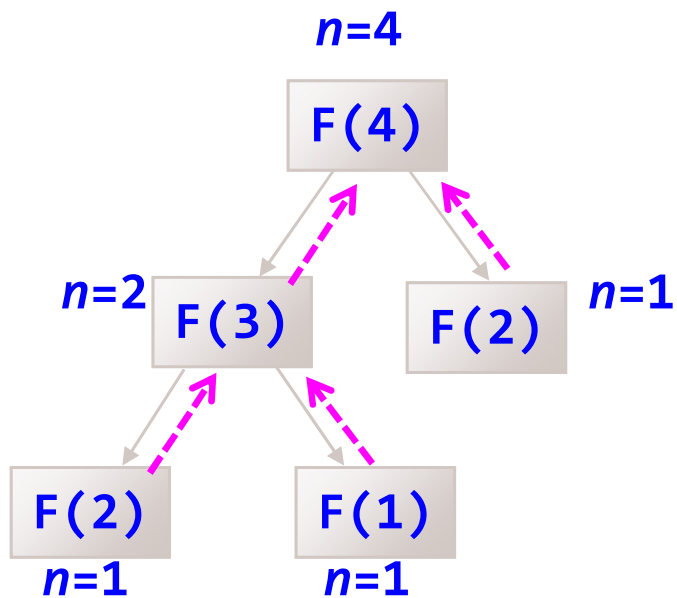
可以用全局变量代替：

```
int s=0;
void add1(int n)
{  if (n==1)
        s=1;
    else
    {    add1(n-1);
        s+=n;
    }
}
```

//全局变量

//理解为:s与add(n)绑定, $s=1+2+\dots+n$

② 递归函数中的非引用形参作为状态变量，可以自动回溯



- n 表示状态
- 状态自动回溯

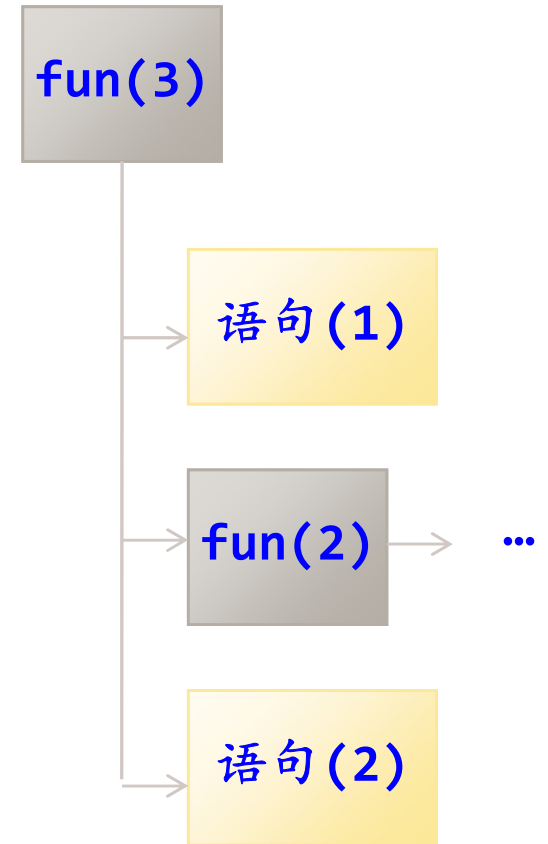


③ 递归调用后面的语句表示该子问题执行完毕后要完成的功能

```
void fun(int n)
{
    if (n>=1)
    {
        printf("n1=%d\n",n);  //(1)
        fun(n-1);
        printf("n2=%d\n",n);  //(2)
    }
}
```

fun(3)的
输出结果

n1=3
n1=2
n1=1
n2=1
n2=2
n2=3



掌握递归函数的执行过程有助于递归算法设计

在**fun(2)**的全部功能
执行后才执行



实验练习

以下两类2选1即可:

➤ **类别1:** P158, 练习题5, 题目: 5、8; P158, 上机实验题5, 题号: 实验题1、2; P160, 题号: 5.leetcode24-两两交换链表中的结点

➤ **类别2:**

✓ Leetcode, 题号: 203、206、3483、394、2487